

Engine Sound Simulator MVP — Detailed Plan

Oct 8, 2026 · @Sanskriti Kumari

Overview and goals

Build a free, browser-based engine sound simulator in about 6 weeks: pick a vehicle, start the engine, rev it, shift gears and hear it respond in real time. It is inspired by mobile apps like RevHeadz but uses only original, free and properly licensed sounds.

The MVP exists to answer one question: do people enjoy revving a virtual engine in the browser enough to come back and share it?

Success criteria for the MVP

- Three playable vehicles: one synthesized engine and two built from free recordings.
- Sound responds to throttle within 50 ms on desktop Chrome and feels smooth, with no clicks or gaps between RPM samples.
- Works on desktop (keyboard) and on mobile (touch) in Chrome, Safari and Firefox.
- First load under 3 seconds on a 4G connection; each vehicle's sounds under 3 MB.
- Total running cost: ₹0 per month.
- Ten friends or community members test it and at least half say they would use it again.

Scope

The MVP covers the core feeling of driving an engine and nothing else; every other idea waits until real users have tried it.

Area	In the MVP	Later
Vehicles	3: synthesized engine, Muscle V8, Inline-4 superbike	Large garage, user-submitted vehicles
Engine controls	Ignition, throttle, brake, gear up/down, rev limiter	Clutch, launch control, burnouts

Area	In the MVP	Later
Sound features	RPM-blended loops, on/off-load layers, backfire pops, shift sounds	Turbo spool, blow-off valve, exhaust customization
Visuals	Tachometer, speedometer, gear indicator	Vehicle images, themes, animated exhaust
Input	Keyboard (desktop), touch slider (mobile)	Gamepad, phone tilt
Pages	Garage (vehicle picker), Drive page per vehicle, Credits	About, blog, per-vehicle SEO content
Accounts and data	None	Accounts, saved setups, leaderboards
Car integration	None	OBD-II connection
Monetization	None	Optional premium vehicle packs

Vehicle names stay generic (for example "Muscle V8", not a brand name) to avoid trademark issues.

Tech stack and tools

Everything below has a free tier or is open source, so the MVP costs nothing to build or run.

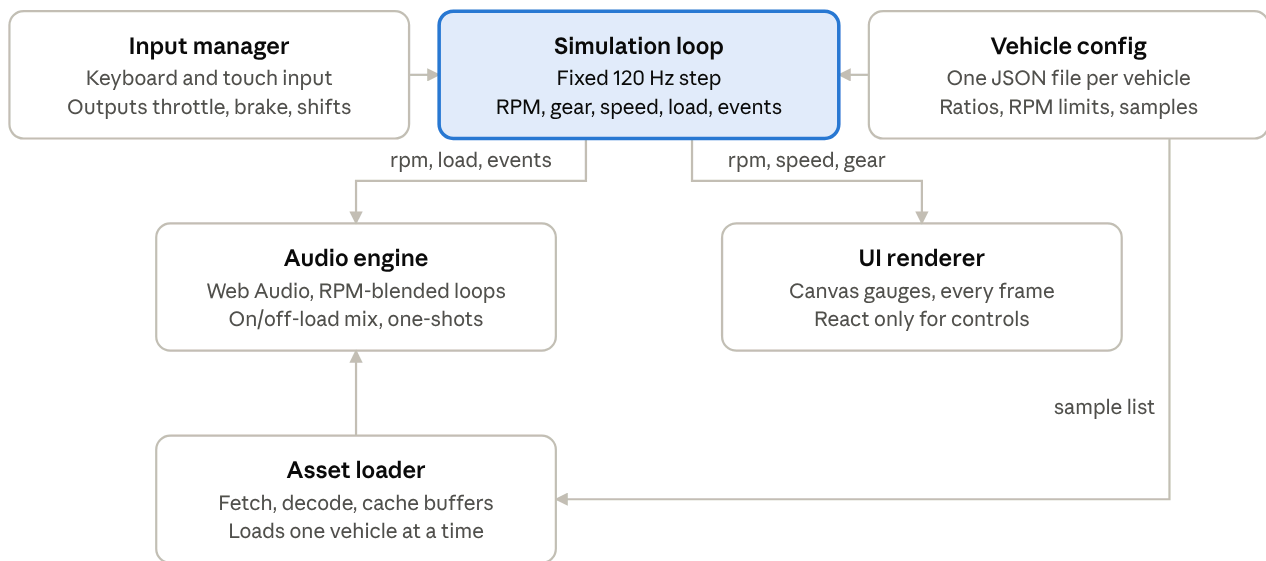
Purpose	Choice	Cost	Notes
Framework	Next.js (App Router) + TypeScript	Free	Static export, no server
Styling	Tailwind CSS	Free	Fast to build responsive controls
Audio	Web Audio API (built into browsers)	Free	No audio library needed
Gauges	HTML canvas	Free	Drawn every frame, outside React
Low-frequency state	useSyncExternalStore (or Zustand)	Free	Gear, ignition, selected vehicle

Purpose	Choice	Cost	Notes
Sound editing	Audacity	Free	Cutting, looping, noise removal
Code hosting	GitHub	Free	Public or private repo
Web hosting	Vercel Hobby	Free	Non-commercial use; Netlify or Cloudflare Pages if monetized later
Analytics	Vercel Web Analytics free tier or Umami (self-hosted)	Free	Just page views and vehicle picks
Testing	Vitest for logic, manual device testing	Free	Simulation math is unit-tested

System architecture

The app has six parts, and only the simulation loop changes engine state.

One simulation loop owns the state; audio and UI only read it



app architecture · 6 parts

Input and the vehicle config feed the simulation loop; the audio engine and the UI read its state, and the asset loader supplies only the selected vehicle's sounds.

Next.js project structure

Engine logic lives in plain TypeScript under `lib/`; React only renders pages and controls, and the simulator page is client-only.

```

engine-sim/
├─ app/
│   ├─ layout.tsx
│   └─ page.tsx                # Garage: vehicle picker (Server
Component)
│   ├─ credits/page.tsx        # Sound credits from credits.json
│   └─ drive/[vehicleId]/page.tsx # Drive page, generateStaticParams per
vehicle
├─ components/
│   ├─ Simulator.tsx           # 'use client'; loaded with dynamic(...,
{ ssr: false })
│   ├─ IgnitionButton.tsx      # creates/resumes AudioContext on tap
│   ├─ ThrottleControl.tsx     # touch slider + keyboard hints
│   ├─ ShiftButtons.tsx
│   ├─ Tachometer.tsx          # canvas, reads state through a ref
│   ├─ Speedometer.tsx
│   └─ GearIndicator.tsx
├─ lib/
│   ├─ input/InputManager.ts
│   ├─ sim/Engine.ts            # RPM, inertia, limiter, load
│   ├─ sim/Gearbox.ts           # ratios, shifting, road speed
│   ├─ sim/Simulation.ts        # fixed-timestep loop, emits events
│   ├─ audio/AudioEngine.ts     # context, master chain
│   ├─ audio/SampleLayer.ts     # RPM crossfade between loops
│   ├─ audio/SynthEngine.ts     # procedural engine (vehicle 1)
│   ├─ audio/OneShots.ts        # starter, shifts, backfires
│   ├─ assets/Loader.ts         # fetch, decode, cache
│   └─ vehicles/index.ts        # reads configs, validates them
├─ hooks/useEngineState.ts      # low-frequency subscription
├─ public/vehicles/
│   ├─ synth_v6/config.json
│   ├─ muscle_v8/config.json + sounds/
│   └─ superbike_i4/config.json + sounds/
├─ credits.json
├─ tests/                      # Vitest unit tests for lib/sim
└─ next.config.ts              # output: 'export'

```

Rules that keep this clean:

- Nothing in `lib/` imports React, so the engine can be tested and reused anywhere.
- High-frequency values (RPM, speed) never go into React state; canvas components read them each animation frame.
- On unmount, `Simulator.tsx` stops the loop, disconnects audio nodes and closes the `AudioContext`.

Vehicle config format

Each vehicle is one `config.json` plus its sound folder, so adding a vehicle never requires code changes.

```
{
  "id": "muscle_v8",
  "name": "Muscle V8",
  "type": "car",
  "engine": {
    "cylinders": 8,
    "idleRpm": 800,
    "redlineRpm": 6500,
    "limiterRpm": 6800,
    "inertia": 0.35,
    "engineBraking": 0.6
  },
  "gearbox": {
    "ratios": [2.97, 2.07, 1.43, 1.0, 0.84, 0.56],
    "finalDrive": 3.42,
    "wheelRadiusM": 0.34,
    "shiftTimeMs": 250
  },
  "audio": {
    "mode": "samples",
    "onLoad": [
      { "file": "on_1000.ogg", "rpm": 1000 },
      { "file": "on_3000.ogg", "rpm": 3000 },
      { "file": "on_5500.ogg", "rpm": 5500 }
    ],
    "offLoad": [
      { "file": "off_1500.ogg", "rpm": 1500 },
      { "file": "off_4000.ogg", "rpm": 4000 }
    ],
    "idle": { "file": "idle_800.ogg", "rpm": 800 },
```

```

    "oneShots": {
      "start": "start.ogg",
      "stop": "stop.ogg",
      "shift": ["shift_1.ogg", "shift_2.ogg"],
      "backfire": ["pop_1.ogg", "pop_2.ogg", "pop_3.ogg"]
    },
    "masterGainDb": -3
  },
  "credits": ["on_3000.ogg: <author>, Freesound, CC0"]
}

```

The synthesized vehicle uses `"mode": "synth"` and replaces the sample lists with tone settings (firing order character, exhaust filter cutoff, roughness).

`lib/vehicles/index.ts` validates every config at build time and fails the build if a referenced sound file is missing.

Simulation design

A fixed-timestep loop at 120 updates per second owns all engine state; audio and UI only read it. Fixed steps keep behaviour identical on a 60 Hz phone and a 144 Hz monitor.

State produced every tick

Field	Meaning
rpm	Current engine speed
throttle	Smoothed 0-1 pedal position
load	0-1, how hard the engine is working (drives on/off-load audio blend)
gear	0 = neutral, 1-6
speedKmh	Road speed from RPM and gearing
ignition	Off, cranking, running
limiterActive	True while bouncing off the limiter

Update rules (in order each tick)

1. Smooth raw input: throttle eases toward the target over about 60 ms so taps don't sound robotic.

2. Engine torque pushes RPM up in proportion to throttle; engine braking and friction pull it down. Inertia from the config sets how quickly it moves. In neutral the engine revs freely; in gear it is tied to road speed.
3. In gear, $\text{RPM} = \text{speed} \times \text{gear ratio} \times \text{final drive} \div \text{wheel circumference}$ (converted to per-minute units); speed changes with torque minus drag and brakes.
4. Rev limiter: above `limiterRpm`, fuel is cut for about 40 ms, producing the familiar bouncing sound.
5. Load = throttle weighted by how much RPM is rising; near 0 when coasting.
6. Events are emitted, not handled here: `start`, `stop`, `shift`, `backfire`, `limiter`.

Event rules

- Backfire: when throttle drops from above 0.6 to below 0.1 while RPM is above 60% of redline, fire 1–4 random pops over the next 600 ms.
- Shift: RPM drops (upshift) or rises (downshift) to match the new gear over `shiftTimeMs`, with a short throttle cut.
- Ignition: cranking lasts about 800 ms, then RPM settles to idle with a small overshoot.

All of `lib/sim` is pure TypeScript and covered by unit tests (for example: RPM never exceeds the limiter, speed is 0 in neutral after braking, backfire triggers only on lift-off).

Audio engine design

Realism comes from blending a few looping recordings by RPM and load, all through the browser's built-in Web Audio API.

Signal chain

Each loop: `AudioBufferSourceNode` (looping) → `GainNode` (crossfade weight) → layer bus. On-load bus and off-load bus → load crossfade → low-pass filter (opens with throttle) → `DynamicsCompressorNode` → master `GainNode` → speakers. One-shots (start, shift, backfire) connect straight to the compressor.

Sample blending (every animation frame)

1. Find the two loops whose recorded RPM sits just below and just above the current RPM.
2. Set each loop's `playbackRate = currentRpm / sampleRpm`, so pitch matches the engine.

3. Crossfade between them with equal-power curves: $\text{weight} = \text{position between the two sample RPMs}$; $\text{gainA} = \cos(\text{weight} \times \pi/2)$, $\text{gainB} = \sin(\text{weight} \times \pi/2)$.
4. Blend the on-load and off-load layers by `load` the same way.
5. Keep all loops running continuously at zero gain when unused; starting and stopping sources causes clicks.

Rules to avoid glitches

- Change every parameter with `setTargetAtTime(value, now, 0.02)`, never by direct assignment, to avoid crackling "zipper" noise.
- Keep playback rates between about 0.6 and 1.6; outside that, recordings sound chipmunk-like or muddy, which tells you to add another sample.
- Limit simultaneous one-shots to 4 and give backfires random pitch ($\pm 8\%$) and volume so repeats don't sound copied.
- Create or resume the `AudioContext` only inside the ignition button's click handler (browser autoplay rules).
- Pause audio when the tab is hidden (`visibilitychange`) and resume on return.

Synthesized engine (vehicle 1)

An oscillator-free approach that sounds closer to a real engine: generate a short pulse per cylinder firing ($\text{rate} = \text{RPM} \div 60 \times \text{cylinders} \div 2$ for four-stroke), shape each pulse with a decaying envelope, add slight random timing jitter, then pass it through band-pass and low-pass filters that model the exhaust. It needs no sound files at all, so it also serves as the instant fallback while recorded samples load.

UI, controls and mobile

The drive screen is one view: gauges at the top, controls at the bottom within thumb reach, and nothing that needs scrolling.

Screens

- Garage: three vehicle cards (name, engine type, short description), tap to drive.
- Drive: tachometer (with redline zone), speedometer, gear indicator, ignition button, throttle, brake, shift buttons, back to garage.
- Credits: list of every sound with author, source link and license, generated from `credits.json`.

Desktop controls

Key	Action
W or Up arrow	Throttle (hold)
S or Down arrow	Brake (hold)
E / Q	Shift up / down
N	Neutral
Space	Quick blip of throttle
I	Ignition on/off

Mobile controls

- Vertical throttle slider on the right that springs back to zero when released; position maps directly to throttle 0–1, so half-throttle is possible.
- Brake pad on the left, shift up/down buttons above it.
- Landscape layout preferred; portrait works with gauges stacked.
- Use `touch-action: none` on controls and pointer events (one code path for mouse and touch), and add light vibration on shifts where supported.

Gauges

Drawn on canvas at the device pixel ratio for sharpness. The needle reads the simulation's RPM every frame with slight smoothing, and the redline zone pulses when the limiter is hit.

Accessibility basics

Visible focus outlines, every control reachable by keyboard, `aria-label` on icon buttons, a master volume slider, and a short "turn your volume down first" note before the first ignition.

Sound sourcing and preparation

Use only CC0 and CC-BY sounds (or your own recordings), and turn each vehicle's raw recordings into 6–10 tagged files.

Sources and license rules

Source	License to use	Credit needed	Notes
Freesound	CC0 or CC-BY only	CC-BY: yes	Filter by license; skip anything marked NC
Sonniss GDC bundles	Royalty-free	No	Professional quality; read each year's license file
Pixabay sound effects	Pixabay content license	No	Fewer specific engines
OpenGameArt	CC0 or CC-BY only	CC-BY: yes	Mostly game-style sounds
Your own recordings	You own them	No	Best quality for specific vehicles

For every file you keep, record the source URL, author and license in `credits.json` on the day you download it.

What to look for

Search terms like "engine idle", "engine rev loop", "V8 steady rpm", "motorcycle engine loop", "car interior engine". The best files hold a steady RPM for several seconds with little background noise. Drive-bys and videos with music or voices are hard to use.

Preparation steps in Audacity (per vehicle)

1. Import the recording and convert to mono, 44.1 kHz.
2. Remove background noise with Noise Reduction (sample a quiet section first).
3. Find steady sections and cut 1–3 second loops: idle, low, mid and high RPM, for on-load (accelerating) and off-load (coasting) where available.
4. Make each loop seamless: cut at zero crossings and apply a short crossfade between the end and the start; test with loop playback.
5. Estimate each loop's RPM (use the spectrum's lowest strong peak: $\text{RPM} \approx \text{firing frequency} \times 120 \div \text{cylinders}$ for four-stroke) and name the file `on_3000.ogg` and so on.
6. Cut one-shots: start, stop, 2 shift sounds, 3 backfire pops.
7. Normalize loops to about -3 dB peak so vehicles have similar loudness.
8. Export as Ogg Vorbis (quality 5) plus an MP3 copy for older Safari. Target under 3 MB per vehicle.

Recording your own (optional for MVP)

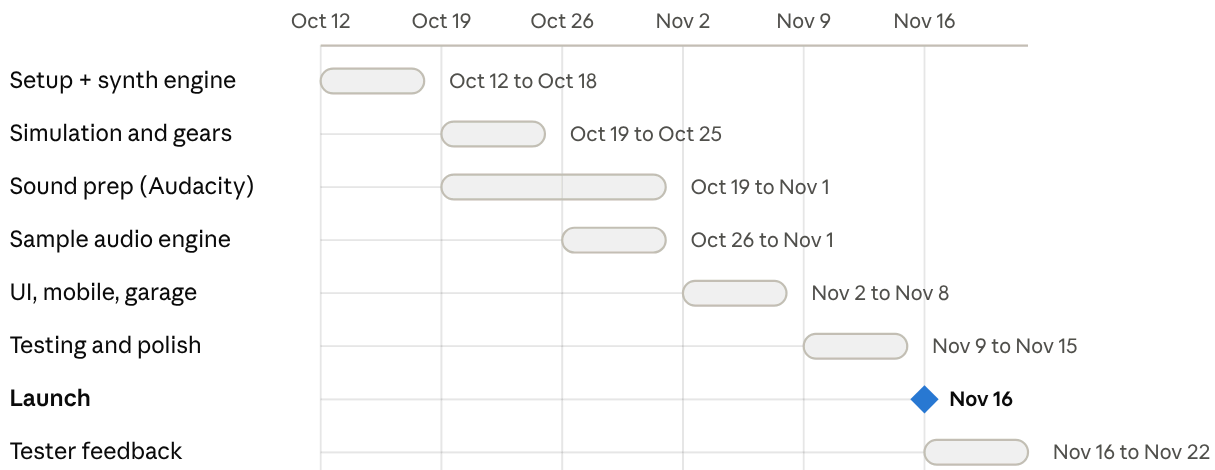
Stationary vehicle, phone or portable recorder with a foam windscreen, about 1 m behind the exhaust and slightly to the side. Record idle, then hold roughly 2000, 3000, 4000 and 5000 RPM for 5 seconds each, followed by sharp revs and lift-offs for pops. Ask the owner's permission and note the date and location.

Roadmap

The MVP ships in six weeks, with sound preparation running alongside the code in weeks 2 and 3.

Six weeks from first commit to launch on Nov 16

Week starts shown on top, assuming work begins Monday Oct 12, 2026



MVP roadmap · 6 weeks

Each week ends with something you can hear or try.

Week 1 — Setup and synthesized engine (goal: an engine that responds to a key)

- ☐ Create the Next.js app with TypeScript, Tailwind and static export
- ☐ Build `InputManager` for the keyboard
- ☐ Basic `Engine.ts` : RPM rises with throttle and falls back to idle
- ☐ `SynthEngine.ts` producing the firing-pulse sound
- ☐ Ignition button that unlocks the AudioContext
- ☐ First deploy to Vercel

Week 2 — Simulation and gears (goal: shifting through 6 gears sounds right)

- ☐ `Gearbox.ts` : ratios, shifting, road speed
- ☐ Fixed-timestep `Simulation.ts` emitting events
- ☐ Rev limiter and backfire logic
- ☐ Vitest tests for `lib/sim`
- ☐ Test audio unlock on a real iPhone

Weeks 2-3 — Sound preparation, in parallel (goal: two vehicles' files ready)

- ☐ Collect candidate recordings from Freesound and Sonnis
- ☐ Pick sources for the Muscle V8 and the Inline-4 superbike; log credits
- ☐ Cut loops and one-shots in Audacity
- ☐ Write both `config.json` files

Week 3 — Sample audio engine (goal: recorded vehicles rev smoothly)

- ☐ `Loader.ts` with decode and cache
- ☐ `SampleLayer.ts` with RPM crossfade
- ☐ On/off-load blend, filter and compressor
- ☐ `OneShots.ts` for start, shifts, backfires
- ☐ Config validation at build time

Week 4 — UI, mobile and garage (goal: fully playable on a phone)

- ☐ Garage page and `drive/[vehicleId]` pages with `generateStaticParams`
- ☐ Canvas tachometer, speedometer and gear indicator
- ☐ Mobile throttle slider, brake and shift buttons, landscape layout
- ☐ Credits page and master volume

Week 5 — Testing and polish (goal: every QA box ticked)

- ☐ Run the Testing and QA checklists
- ☐ Fix clicks, loudness balance and mobile performance
- ☐ Add analytics, page metadata and share images

Week 6 — Launch and feedback (launch on `Nov 16, 2026`)

- ☐ Share with 10 testers
- ☐ Collect feedback for a week, then choose the next features

Testing and QA

Automated tests cover the simulation math; sound and feel are tested by ear on real devices before launch.

Automated (Vitest)

- ☐ RPM stays between idle and limiter in every gear
- ☐ Speed matches the gearing formula within 1%
- ☐ Upshift lowers RPM, downshift raises it, neutral frees the engine
- ☐ Backfire fires only on high-RPM lift-off
- ☐ Every vehicle config is valid and all referenced sound files exist
- ☐ Crossfade weights always sum to constant power

Manual, by ear

- ☐ No clicks or pops while sweeping slowly from idle to redline
- ☐ No audible jump when passing between two sample RPMs
- ☐ Throttle feels instant (tap and release)
- ☐ Limiter bounce sounds natural, not a stutter
- ☐ Vehicles have similar loudness
- ☐ Leaving the drive page fully stops the sound

Devices

- ☐ Desktop: Chrome, Firefox, Safari (Mac), Edge
- ☐ Android: Chrome on a mid-range phone
- ☐ iPhone: Safari, including silent-switch behaviour
- ☐ Tablet in landscape

Performance

- ☐ Steady 60 fps on a mid-range Android phone
- ☐ Lighthouse performance score of 90 or more on the garage page
- ☐ First vehicle playable within 3 seconds on throttled 4G

Deployment and launch

Deploy as a static site on Vercel's free Hobby plan from day one, so every push gets a live preview link to share with testers.

Setup

1. Push the repo to GitHub.
2. Import it in Vercel; it detects Next.js automatically.
3. In `next.config.ts` set `output: 'export'` and `images: { unoptimized: true }`.
4. Add cache headers so files under `/vehicles/` are cached for a long time (rename files when they change).
5. Use the free `*.vercel.app` address for the MVP; a custom domain (around ₹800–1,000 per year) is the only optional cost.

Launch checklist

- ☐ Credits page complete and linked from the footer
- ☐ Page titles, descriptions and a share image for each vehicle page
- ☐ Basic analytics: visits, vehicle picked, ignition pressed, time on drive page
- ☐ A simple feedback link (Google Form or GitHub Issues)
- ☐ Share with 10 testers, then car and bike communities (Reddit, Instagram, local Bengaluru groups)

Risks and mitigations

The biggest risk is sound quality from free recordings; the synthesized engine guarantees a working MVP even if recordings disappoint.

Risk	Impact	Mitigation
Free recordings don't loop cleanly or lack steady RPMs	Engine sounds fake or jumpy	Try Sonniss packs first; use fewer, better loops; fall back to the synth engine; record your own later
Clicks and zipper noise	Feels cheap	Always-running loops, <code>setTargetAtTime</code> ramps, zero-crossing cuts
iOS Safari audio quirks (silent switch, autoplay)	No sound on iPhones	Start audio only from a tap; test on a real iPhone early (week 2)
Mobile performance	Stutter, lag	Max 6 active loops, canvas-only gauges, no React re-renders per frame

Risk	Impact	Mitigation
Licensing mistakes	Takedown requests	CC0/CC-BY only, credits logged at download time, no NC files
Brand names in vehicle names	Trademark complaints	Generic descriptive names only
Scope creep	MVP never ships	Anything outside the Scope table goes to a "later" list
Vercel Hobby is non-commercial	Must move if monetized	Static export moves to Netlify or Cloudflare Pages in minutes

After the MVP

Pick the next features based on what testers actually use; these are the likely candidates in rough priority order.

1. More vehicles, especially your own recordings of local cars and bikes.
2. Turbo spool and blow-off valve layers.
3. PWA install and offline play.
4. Exhaust and tuning options (straight pipe, muffled, crackle map).
5. Gamepad support and phone-tilt throttle.
6. Accounts, saved setups and sharing a setup by link.
7. OBD-II connection through Web Bluetooth (supported only in Chromium browsers).
8. Optional premium vehicle packs, which would require moving off Vercel Hobby.